

LLD & HLD Interview Handbook

CRED-style Machine Coding & System Design PYQs

Prepared for Jatin

July 30, 2026

Contents

1	Part 0: Foundations	2
1.1	What is LLD (Low Level Design)?	2
1.2	What is HLD (High Level Design)?	2
1.3	The Universal LLD Framework (follow this every time)	2
1.4	The Universal HLD Framework (follow this every time)	2
1.5	SOLID Principles (one-line memory hooks)	3
1.6	Design Patterns You Will Actually Use	3
2	LLD Problems	4
2.1	LLD 1: Design a Parking Lot System	4
2.1.1	Requirements	4
2.1.2	Core Classes	4
2.1.3	Code Skeleton	4
2.2	LLD 2: Design a Contact Management System	6
2.2.1	Requirements	6
2.2.2	Core Classes	7
2.2.3	Code Skeleton (core: Trie-based search)	7
2.3	LLD 3: Design an Idempotent Payment Processing System	9
2.3.1	Requirements	9
2.3.2	Core Classes	9
2.3.3	Code Skeleton	9
2.4	LLD 4: Design a CRED Coins Rewards System	11
2.4.1	Requirements	12
2.4.2	Core Classes	12
2.4.3	Code Skeleton	12
2.5	LLD 5: Design a Bill Due-Date Reminder & Notification System	14
2.5.1	Requirements	14
2.5.2	Core Classes	14
2.5.3	Code Skeleton	14

3	HLD Problems	17
3.1	HLD 1: Design CRED's Credit Card Bill Payment System	17
3.1.1	Functional Requirements	17
3.1.2	Non-Functional Requirements	17
3.1.3	Capacity Estimation (back of envelope)	17
3.1.4	API Design	17
3.1.5	Database Schema (high level)	17
3.1.6	High-Level Architecture	18
3.1.7	Deep Dive: Idempotent, Exactly-Once Payment Flow	18
3.2	HLD 2: Design a De-duplication Service for Duplicate Payments	19
3.2.1	Functional Requirements	19
3.2.2	Non-Functional Requirements	19
3.2.3	Capacity Estimation	19
3.2.4	API Design	19
3.2.5	High-Level Architecture	20
3.2.6	Deep Dive: Atomic Check-and-Reserve	20
3.3	HLD 3: Design a Financial Health / Credit Score Aggregation System	20
3.3.1	Functional Requirements	20
3.3.2	Non-Functional Requirements	21
3.3.3	Capacity Estimation	21
3.3.4	API Design	21
3.3.5	Database Schema	21
3.3.6	High-Level Architecture	21
3.3.7	Deep Dive: Isolating Unreliable External Bureau APIs	22
4	Quick Revision Cheat-Sheet	23
4.1	LLD Pattern Map	23
4.2	HLD Recurring Themes at CRED	23
4.3	Interview Delivery Tips	23

1 Part 0: Foundations

1.1 What is LLD (Low Level Design)?

LLD is about designing the **classes, interfaces, relationships, and methods** for a specific software component. You are given a real-world problem (“design a parking lot”) and must produce working, extensible object-oriented code within 45–90 minutes.

What interviewers evaluate:

- Requirement clarification (do you ask questions before jumping to code?)
- Correct use of OOP: abstraction, encapsulation, inheritance, polymorphism
- SOLID principles applied naturally, not forced
- Design patterns used where they genuinely fit (Strategy, Factory, Observer, Singleton, State)
- Extensibility: can you add a new feature without rewriting everything?
- Working, compilable code – not just diagrams

1.2 What is HLD (High Level Design)?

HLD is about designing a **large-scale distributed system**. You are not writing classes; you are drawing boxes and arrows: services, databases, queues, caches, load balancers – and justifying every choice with numbers (scale, latency, consistency needs).

What interviewers evaluate:

- Requirement gathering: functional + non-functional (scale, latency, consistency vs availability)
- Back-of-envelope capacity estimation (QPS, storage, bandwidth)
- API design
- Database choice and schema (SQL vs NoSQL, and why)
- High-level architecture diagram (services, queues, caches)
- Deep dive into the hardest sub-problem (this is where CRED digs in – e.g., idempotency, deduplication)
- Trade-offs: CAP theorem, consistency models, failure handling

1.3 The Universal LLD Framework (follow this every time)

1. **Clarify requirements** – list functional requirements out loud (2–3 min)
2. **Identify core entities/classes** – nouns in the problem become classes
3. **Identify relationships** – has-a (composition), is-a (inheritance)
4. **Identify behaviors** – verbs become methods; varying behavior becomes an interface (Strategy pattern)
5. **Draw it out** – class diagram (boxes with fields/methods, arrows for relationships)
6. **Code the skeleton** – interfaces first, then concrete classes, then the orchestrator/manager class
7. **Handle edge cases and extensions** – interviewer will ask “what if we add X?”

1.4 The Universal HLD Framework (follow this every time)

1. **Functional requirements** – what must the system do (3–5 bullet points, agree scope with interviewer)
2. **Non-functional requirements** – scale (users, QPS), latency targets, consistency vs availability, durability

3. **Capacity estimation** – rough math: requests/sec, storage/year, bandwidth
4. **API design** – 3–5 core REST/gRPC endpoints with request/response shape
5. **Database schema** – tables/collections, choose SQL (strong consistency, relations) vs NoSQL (scale, flexible schema)
6. **High level architecture** – client → load balancer → API gateway → services → DB/-cache/queue
7. **Deep dive** – pick the hardest part (interviewer usually asks) and go deep: idempotency, dedup, consistency, race conditions
8. **Bottlenecks & scaling** – sharding, replication, caching, async processing, circuit breakers

1.5 SOLID Principles (one-line memory hooks)

- **S – Single Responsibility:** A class should have one reason to change.
- **O – Open/Closed:** Open for extension, closed for modification (use interfaces, not if-else chains).
- **L – Liskov Substitution:** A subclass should be usable wherever the base class is expected.
- **I – Interface Segregation:** Don't force classes to implement methods they don't need.
- **D – Dependency Inversion:** Depend on abstractions (interfaces), not concrete classes.

1.6 Design Patterns You Will Actually Use

- **Strategy** – interchangeable algorithms (pricing strategy, payment routing strategy)
- **Factory** – object creation decoupled from usage (creating vehicle objects, notification objects)
- **Observer** – one-to-many notification (notify users when bill is due, notify slots freed)
- **Singleton** – exactly one instance (ParkingLot manager, system config)
- **State** – object behavior changes with internal state (order status, payment status)
- **Decorator** – add responsibilities dynamically (reward multipliers, discount stacking)

2 LLD Problems

2.1 LLD 1: Design a Parking Lot System

Difficulty: Medium

2.1.1 Requirements

- Multiple floors, each with multiple spots of different sizes (small, medium, large, handicapped)
- Multiple vehicle types (motorcycle, car, bus) – a vehicle can only park in a compatible spot
- Entry gate assigns nearest available compatible spot; exit gate computes fee
- Fee depends on duration and vehicle type
- Support querying free spot count per floor

Key Idea

The core insight: don't hardcode "if car then medium spot" logic. Use a **Strategy** for spot allocation and a **Strategy** for fee calculation, so both can be swapped without touching core classes.

2.1.2 Core Classes

- **Vehicle** (abstract) → **Car**, **Bike**, **Truck**
- **ParkingSpot** – holds spot id, size, occupied status, current vehicle
- **Floor** – holds list of **ParkingSpot**
- **ParkingLot** (Singleton) – holds list of **Floor**, orchestrates entry/exit
- **Ticket** – entry time, spot id, vehicle
- **FeeStrategy** (interface) → **HourlyFeeStrategy**, **FlatFeeStrategy**
- **SpotAssignmentStrategy** (interface) → **NearestSpotStrategy**

2.1.3 Code Skeleton

```
enum class VehicleType { MOTORCYCLE, CAR, BUS };
enum class SpotSize { SMALL, MEDIUM, LARGE };

class Vehicle {
public:
    string licensePlate;
    VehicleType type;
    Vehicle(string plate, VehicleType t) : licensePlate(plate), type(t) {}
};

class ParkingSpot {
public:
    int spotId;
    SpotSize size;
    bool isOccupied = false;
    Vehicle* currentVehicle = nullptr;

    bool canFit(VehicleType vt) {
        if (vt == VehicleType::MOTORCYCLE) return true;
    }
};
```

```
        if (vt == VehicleType::CAR) return size != SpotSize::SMALL;
        return size == SpotSize::LARGE; // bus needs large
    }
};

class Floor {
public:
    int floorNumber;
    vector<ParkingSpot*> spots;

    ParkingSpot* findAvailableSpot(VehicleType vt) {
        for (auto* spot : spots)
            if (!spot->isOccupied && spot->canFit(vt))
                return spot;
        return nullptr;
    }
};

// Strategy Pattern for fee calculation
class FeeStrategy {
public:
    virtual double calculateFee(long entryTime, long exitTime, VehicleType vt) = 0;
    virtual ~FeeStrategy() = default;
};

class HourlyFeeStrategy : public FeeStrategy {
public:
    double calculateFee(long entryTime, long exitTime, VehicleType vt) override {
        double hours = ceil((exitTime - entryTime) / 3600.0);
        double rate = (vt == VehicleType::CAR) ? 20.0 : (vt == VehicleType::BUS ? 50.0 :
            10.0);
        return hours * rate;
    }
};

class Ticket {
public:
    int ticketId;
    Vehicle* vehicle;
    ParkingSpot* spot;
    long entryTime;
};

class ParkingLot {
private:
    static ParkingLot* instance;
    vector<Floor*> floors;
    FeeStrategy* feeStrategy;
    unordered_map<int, Ticket*> activeTickets;
    int ticketCounter = 0;

    ParkingLot() { feeStrategy = new HourlyFeeStrategy(); }

public:
```

```

static ParkingLot* getInstance() {
    if (!instance) instance = new ParkingLot();
    return instance;
}

Ticket* parkVehicle(Vehicle* v) {
    for (auto* floor : floors) {
        ParkingSpot* spot = floor->findAvailableSpot(v->type);
        if (spot) {
            spot->isOccupied = true;
            spot->currentVehicle = v;
            Ticket* t = new Ticket(++ticketCounter, v, spot, time(nullptr));
            activeTickets[t->ticketId] = t;
            return t;
        }
    }
    return nullptr; // lot full
}

double exitVehicle(int ticketId) {
    Ticket* t = activeTickets[ticketId];
    double fee = feeStrategy->calculateFee(t->entryTime, time(nullptr), t->vehicle->
        type);
    t->spot->isOccupied = false;
    t->spot->currentVehicle = nullptr;
    activeTickets.erase(ticketId);
    return fee;
}
};
ParkingLot* ParkingLot::instance = nullptr;

```

Trade-offs & Follow-ups

”What if lot is full?” → maintain a waiting queue, or reject with a clear response.
”What if we want dynamic pricing (surge)?” → swap `FeeStrategy` implementation, no other code changes – this is exactly why Strategy pattern was used.
”Multiple entry gates concurrently?” → spot allocation must be thread-safe; use a mutex around `findAvailableSpot` + assignment, or per-floor locks to reduce contention.

2.2 LLD 2: Design a Contact Management System

Difficulty: Hard

2.2.1 Requirements

- Add, edit, delete contacts (name, phone numbers, emails, groups)
- Search contacts by name prefix, phone, or group – fast search is the crux
- Group contacts (Favorites, Work, Family) – a contact can belong to multiple groups
- Merge duplicate contacts
- Sort/list contacts alphabetically

Key Idea

The "search" requirement is the crux of this problem. Naive $O(n)$ linear scan is what most candidates default to – the interviewer wants you to reach for a **Trie** for prefix search on names, plus a **HashMap** for exact lookups (phone $\rightarrow$ contact), giving $O(k)$ search where k = query length.

2.2.2 Core Classes

- **Contact** – id, name, list of phone numbers, list of emails, set of groupIds
- **Group** – id, name, set of contactIds
- **TrieNode** / **Trie** – for prefix search on contact names
- **ContactManager** – orchestrator: add/edit/delete/search/merge
- **SearchStrategy** (interface) $\rightarrow$ **NameSearch**, **PhoneSearch**, **GroupSearch**

2.2.3 Code Skeleton (core: Trie-based search)

```
class Contact {
public:
    int id;
    string name;
    vector<string> phones;
    vector<string> emails;
    unordered_set<int> groupIds;
};

class TrieNode {
public:
    unordered_map<char, TrieNode*> children;
    unordered_set<int> contactIds; // contacts whose name passes through this node
};

class Trie {
    TrieNode* root = new TrieNode();
public:
    void insert(const string& name, int contactId) {
        TrieNode* cur = root;
        for (char c : name) {
            if (!cur->children.count(c)) cur->children[c] = new TrieNode();
            cur = cur->children[c];
            cur->contactIds.insert(contactId); // supports prefix search
        }
    }

    void remove(const string& name, int contactId) {
        TrieNode* cur = root;
        for (char c : name) {
            if (!cur->children.count(c)) return;
            cur = cur->children[c];
            cur->contactIds.erase(contactId);
        }
    }
}
```

```
vector<int> searchPrefix(const string& prefix) {
    TrieNode* cur = root;
    for (char c : prefix) {
        if (!cur->children.count(c)) return {};
        cur = cur->children[c];
    }
    return vector<int>(cur->contactIds.begin(), cur->contactIds.end());
}

};

class ContactManager {
private:
    unordered_map<int, Contact*> contacts;
    unordered_map<string, int> phoneToContactId; // O(1) phone lookup
    unordered_map<int, Group*> groups;
    Trie nameTrie;
    int idCounter = 0;

public:
    int addContact(string name, vector<string> phones) {
        Contact* c = new Contact{++idCounter, name, phones, {}, {}};
        contacts[c->id] = c;
        nameTrie.insert(name, c->id);
        for (auto& p : phones) phoneToContactId[p] = c->id;
        return c->id;
    }

    vector<Contact*> searchByPrefix(string prefix) {
        vector<int> ids = nameTrie.searchPrefix(prefix);
        vector<Contact*> result;
        for (int id : ids) result.push_back(contacts[id]);
        return result;
    }

    Contact* searchByPhone(string phone) {
        if (phoneToContactId.count(phone)) return contacts[phoneToContactId[phone]];
        return nullptr;
    }

    void mergeContacts(int id1, int id2) {
        Contact* c1 = contacts[id1];
        Contact* c2 = contacts[id2];
        for (auto& p : c2->phones) { c1->phones.push_back(p); phoneToContactId[p] = id1; }
        for (auto& e : c2->emails) c1->emails.push_back(e);
        for (auto gid : c2->groupIds) c1->groupIds.insert(gid);
        contacts.erase(id2);
        nameTrie.remove(c2->name, id2);
    }
};
```

Trade-offs & Follow-ups

”**Search must be fuzzy / typo-tolerant?**” → Trie alone won’t help; mention Levenshtein-distance based fuzzy matching or a search index (Elasticsearch) at scale – but for LLD scope, Trie is the expected answer.

”**Millions of contacts, single machine?**” → this becomes an HLD question (sharding by first letter / user id).

”**Why not just sort + binary search?**” → binary search only supports exact prefix boundaries efficiently if re-sorted on every insert; Trie gives incremental $O(k)$ insert/search.

2.3 LLD 3: Design an Idempotent Payment Processing System

Difficulty: Hard – CRED’s most-asked LLD

2.3.1 Requirements

- Client initiates a payment; network can retry the same request (client timeout, mobile network drop)
- The system must guarantee a payment is **never charged twice** even if the same request arrives multiple times
- Support multiple payment methods (UPI, card, wallet) via a common interface
- Return the same result for a duplicate request instead of reprocessing

Key Idea

The client generates a unique **Idempotency Key** (UUID) per logical payment attempt and sends it with every retry. The server uses this key, not the request body, as the deduplication signal. First request with a key: process and store (**key** → **result**). Any repeat of that key: short-circuit and return the stored result – never touch the payment gateway again.

2.3.2 Core Classes

- **IdempotencyKey** – string (UUID), tied to one logical operation
- **PaymentRequest** – idempotencyKey, amount, source, destination, paymentMethod
- **PaymentStatus** enum – PENDING, SUCCESS, FAILED
- **IdempotencyStore** – maps key → **PaymentResult** (with status, must be atomic on insert)
- **PaymentGateway** (interface) → **UpiGateway**, **CardGateway**, **WalletGateway** (Strategy)
- **PaymentProcessor** – orchestrator with locking around the check-then-act

2.3.3 Code Skeleton

```
enum class PaymentStatus { PENDING, SUCCESS, FAILED };

class PaymentResult {
public:
    string idempotencyKey;
    PaymentStatus status;
    string transactionId;
};
```

```
class PaymentRequest {
public:
    string idempotencyKey;
    double amount;
    string source, destination;
};

// Strategy for different payment rails
class PaymentGateway {
public:
    virtual string charge(double amount, string src, string dst) = 0; // returns txnId
    virtual ~PaymentGateway() = default;
};

class UpiGateway : public PaymentGateway {
public:
    string charge(double amount, string src, string dst) override {
        // call external UPI network
        return "UPI-TXN-" + to_string(rand());
    }
};

// Thread-safe idempotency store: key -> in-flight or completed result
class IdempotencyStore {
private:
    unordered_map<string, PaymentResult> store;
    mutex mtx;

public:
    // Returns true if this is a NEW key (caller should process),
    // false if key already exists (caller should return existing result)
    bool tryInsertPending(const string& key, PaymentResult& outExisting) {
        lock_guard<mutex> lock(mtx);
        if (store.count(key)) {
            outExisting = store[key];
            return false; // duplicate
        }
        store[key] = PaymentResult{key, PaymentStatus::PENDING, ""};
        return true; // new, proceed
    }

    void updateResult(const string& key, PaymentStatus status, string txnId) {
        lock_guard<mutex> lock(mtx);
        store[key] = PaymentResult{key, status, txnId};
    }

    PaymentResult get(const string& key) {
        lock_guard<mutex> lock(mtx);
        return store[key];
    }
};

class PaymentProcessor {
```

```

private:
    IdempotencyStore idempotencyStore;
    PaymentGateway* gateway;

public:
    PaymentProcessor(PaymentGateway* g) : gateway(g) {}

    PaymentResult processPayment(PaymentRequest req) {
        PaymentResult existing;
        bool isNew = idempotencyStore.tryInsertPending(req.idempotencyKey, existing);

        if (!isNew) {
            // Duplicate request: if a previous attempt is still PENDING,
            // in production you'd poll/wait; for LLD scope, return existing state.
            return existing;
        }

        // First time seeing this key -- actually charge
        try {
            string txnId = gateway->charge(req.amount, req.source, req.destination);
            idempotencyStore.updateResult(req.idempotencyKey, PaymentStatus::SUCCESS,
                                          txnId);
            return idempotencyStore.get(req.idempotencyKey);
        } catch (...) {
            idempotencyStore.updateResult(req.idempotencyKey, PaymentStatus::FAILED, "");
            return idempotencyStore.get(req.idempotencyKey);
        }
    }
};

```

Trade-offs & Follow-ups

Why not dedupe on (amount, source, dest, timestamp)? → Two genuinely different payments can have identical amount/source/dest within the same second (e.g. two separate Rs.100 transfers) – content-based dedup causes false positives. A client-generated UUID avoids this entirely.

Race condition: two retries arrive at the exact same instant? → tryInsertPending must be atomic (mutex here; in a distributed system, a DB unique constraint on idempotency_key or Redis SETNX achieves the same).

What if the first request is still processing when the retry comes in? → Real systems return HTTP 409/425 telling client to retry later, or block briefly with a short poll – this is a great “what would you improve” follow-up answer.

Persistence: the idempotency key must survive process crashes – backed by DB row, not just in-memory map (this in-memory map is for LLD demonstration only).

2.4 LLD 4: Design a CRED Coins Rewards System

Difficulty: Medium

2.4.1 Requirements

- Users earn CRED coins on bill payments (based on rules: base rate + bonus multipliers)
- Coins can be redeemed against a catalog of rewards (vouchers, cashback)
- Multiple concurrent rule types: flat rate, percentage of amount, streak bonus, first-time bonus
- Coins have an expiry
- Prevent double-crediting for the same transaction (same idempotency theme as payments!)

Key Idea

Reward calculation is inherently "pluggable rules stacked together" – this is a textbook **Chain of Responsibility** / **Decorator** scenario: each rule wraps and adds to the base coin calculation, rather than one giant if-else block.

2.4.2 Core Classes

- **Wallet** – userId, coinBalance, list of **Transaction**
- **CoinTransaction** – id, type (EARN/REDEEM), amount, sourceBillTxnId (for idempotency), expiryDate
- **RewardRule** (interface) → **BaseRateRule**, **StreakBonusRule**, **FirstTimeBonusRule** (Decorator/Chain)
- **RewardCatalogItem** – voucher/cashback, coin cost
- **RewardsEngine** – orchestrator

2.4.3 Code Skeleton

```
class CoinCalculationContext {
public:
    double billAmount;
    int userStreakDays;
    bool isFirstPayment;
};

// Decorator-style stacking of rules
class RewardRule {
public:
    virtual int calculateCoins(CoinCalculationContext& ctx, int baseCoins) = 0;
    virtual ~RewardRule() = default;
};

class BaseRateRule : public RewardRule {
public:
    int calculateCoins(CoinCalculationContext& ctx, int baseCoins) override {
        return (int)(ctx.billAmount * 0.01); // 1% of bill as coins
    }
};

class StreakBonusRule : public RewardRule {
    RewardRule* wrappee;
public:
    StreakBonusRule(RewardRule* w) : wrappee(w) {}
};
```

```
    int calculateCoins(CoinCalculationContext& ctx, int baseCoins) override {
        int coins = wrappee->calculateCoins(ctx, baseCoins);
        if (ctx.userStreakDays >= 3) coins += 50; // streak bonus
        return coins;
    }
};

class FirstTimeBonusRule : public RewardRule {
    RewardRule* wrappee;
public:
    FirstTimeBonusRule(RewardRule* w) : wrappee(w) {}
    int calculateCoins(CoinCalculationContext& ctx, int baseCoins) override {
        int coins = wrappee->calculateCoins(ctx, baseCoins);
        if (ctx.isFirstPayment) coins += 200;
        return coins;
    }
};

class Wallet {
public:
    int userId;
    long coinBalance = 0;
    unordered_set<string> creditedBillTxnIds; // idempotency guard
};

class RewardsEngine {
private:
    unordered_map<int, Wallet*> wallets;

public:
    bool creditCoins(int userId, string billTxnId, CoinCalculationContext ctx) {
        Wallet* w = wallets[userId];
        if (w->creditedBillTxnIds.count(billTxnId)) return false; // already credited

        RewardRule* rule = new FirstTimeBonusRule(new StreakBonusRule(new BaseRateRule()))
        ;
        int coins = rule->calculateCoins(ctx, 0);

        w->coinBalance += coins;
        w->creditedBillTxnIds.insert(billTxnId);
        return true;
    }

    bool redeemCoins(int userId, int coinCost) {
        Wallet* w = wallets[userId];
        if (w->coinBalance < coinCost) return false;
        w->coinBalance -= coinCost;
        return true;
    }
};
```

Trade-offs & Follow-ups

Coin expiry? → Store coins in expiry-tagged batches (like a min-heap by expiry date) and deduct oldest-first (FIFO) on redemption – a background job periodically sweeps expired batches.

Concurrent redemption from two devices? → same idempotency + locking discipline as the Payment system – balance check-and-deduct must be atomic (mutex or DB row lock with `SELECT ... FOR UPDATE`).

Adding a new bonus rule later? → Just wrap another Decorator – Open/Closed principle in action, no existing rule class touched.

2.5 LLD 5: Design a Bill Due-Date Reminder & Notification System

Difficulty: Medium

2.5.1 Requirements

- Each bill has a due date; system must remind users at T-3 days, T-1 day, and on due date
- Support multiple notification channels: push, SMS, email – user can opt into any subset
- Avoid sending duplicate reminders if the job runs more than once for the same window
- Should scale to scanning millions of bills daily without a full table scan each time

Key Idea

This is a classic **Observer pattern** (bill status changes → notify all subscribed channels) combined with a **scheduling** design: don't scan every bill every run – bucket bills by due-date so each day's job only touches that day's bucket.

2.5.2 Core Classes

- **Bill** – id, userId, dueDate, amount, status
- **NotificationChannel** (interface, Observer) → **PushChannel**, **SmsChannel**, **EmailChannel**
- **ReminderScheduler** – subject; holds list of Bills bucketed by due date, notifies channels
- **ReminderRule** – defines offsets (T-3, T-1, T-0)
- **SentReminderLog** – (billId, offsetType) → prevents duplicate sends (idempotency again!)

2.5.3 Code Skeleton

```
enum class ReminderOffset { T_MINUS_3, T_MINUS_1, T_0 };

class Bill {
public:
    int billId;
    int userId;
    long dueDateEpoch;
    double amount;
};

// Observer pattern: channels subscribe to reminder events
class NotificationChannel {
```

```

public:
    virtual void send(int userId, string message) = 0;
    virtual ~NotificationChannel() = default;
};

class PushChannel : public NotificationChannel {
public:
    void send(int userId, string message) override {
        cout << "Push to user " << userId << ": " << message << endl;
    }
};

class SmsChannel : public NotificationChannel {
public:
    void send(int userId, string message) override {
        cout << "SMS to user " << userId << ": " << message << endl;
    }
};

class ReminderScheduler {
private:
    // bucket bills by due date -> avoids full scan; each day's cron only reads relevant
    // buckets
    unordered_map<long, vector<Bill*>> billsByDueDate;
    unordered_map<int, vector<NotificationChannel*>> userChannels; // opted-in channels
    set<pair<int, ReminderOffset>> sentLog; // idempotency guard: (billId, offset)

public:
    void addBill(Bill* b) {
        billsByDueDate[b->dueDateEpoch].push_back(b);
    }

    void subscribeChannel(int userId, NotificationChannel* channel) {
        userChannels[userId].push_back(channel);
    }

    // Called by a daily cron job with "today"
    void runDailyJob(long todayEpoch) {
        checkAndNotify(todayEpoch + 3 * 86400, ReminderOffset::T_MINUS_3, "Bill due in 3
            days");
        checkAndNotify(todayEpoch + 1 * 86400, ReminderOffset::T_MINUS_1, "Bill due
            tomorrow");
        checkAndNotify(todayEpoch, ReminderOffset::T_0, "Bill due today");
    }

private:
    void checkAndNotify(long targetDate, ReminderOffset offset, string message) {
        if (!billsByDueDate.count(targetDate)) return; // O(1) bucket lookup, no full scan

        for (Bill* bill : billsByDueDate[targetDate]) {
            auto key = make_pair(bill->billId, offset);
            if (sentLog.count(key)) continue; // already sent -- idempotent

            for (auto* channel : userChannels[bill->userId])

```

```
        channel->send(bill->userId, message);  
        sentLog.insert(key);  
    }  
}  
};
```

Trade-offs & Follow-ups

Millions of bills, cron job at scale? → At LLD scope this is fine; at HLD scope you'd shard the bucket by (dueDate, userId hash) across workers, or use a delayed-message queue (e.g., publish a message with a future visibility timeout) instead of scanning.

User changes notification preference mid-flow? → Observer subscription list is looked up live at send time, so it naturally picks up the latest preference.

Why bucket by date instead of a priority queue by due date? → A min-heap works too and is arguably cleaner for "next N reminders due" queries; bucketing wins when the daily cron pattern is fixed and you want $O(1)$ day lookup.

3 HLD Problems

3.1 HLD 1: Design CRED's Credit Card Bill Payment System

Difficulty: Hard

3.1.1 Functional Requirements

- User links credit cards, views bill amount and due date
- User pays bill via UPI/card/netbanking through CRED
- Payment status must be reliably tracked (pending, success, failed) and reconciled with the bank
- User earns CRED coins on successful payment
- User gets payment confirmation and receipt

3.1.2 Non-Functional Requirements

- High consistency for money movement (favor consistency over availability for the payment write path – CP in CAP)
- High availability for read paths (viewing bill status)
- Durability: no payment record can ever be lost
- Idempotency: retries must not double-charge
- Scale: assume 10M users, peak traffic around month-end bill due dates

3.1.3 Capacity Estimation (back of envelope)

- 10M users $\times$ 1.5 cards average $\rightarrow$ 15M bills/month
- Payments cluster near due dates: assume 30% of bills paid in a 3-day peak window $\rightarrow$ 4.5M payments / 3 days $\approx$ 17 payments/sec average, with bursts to $\sim$ 200–500/sec at peak hours
- Each payment record $\approx$ 1KB $\rightarrow$ 15M $\times$ 1KB $\approx$ 15GB/month of payment data (small; DB choice is about consistency, not raw volume here)

3.1.4 API Design

```
POST /api/v1/payments/initiate
Body: { idempotencyKey, billId, amount, paymentMethod }
Response: { paymentId, status: PENDING }

GET /api/v1/payments/{paymentId}/status
Response: { paymentId, status, transactionId }

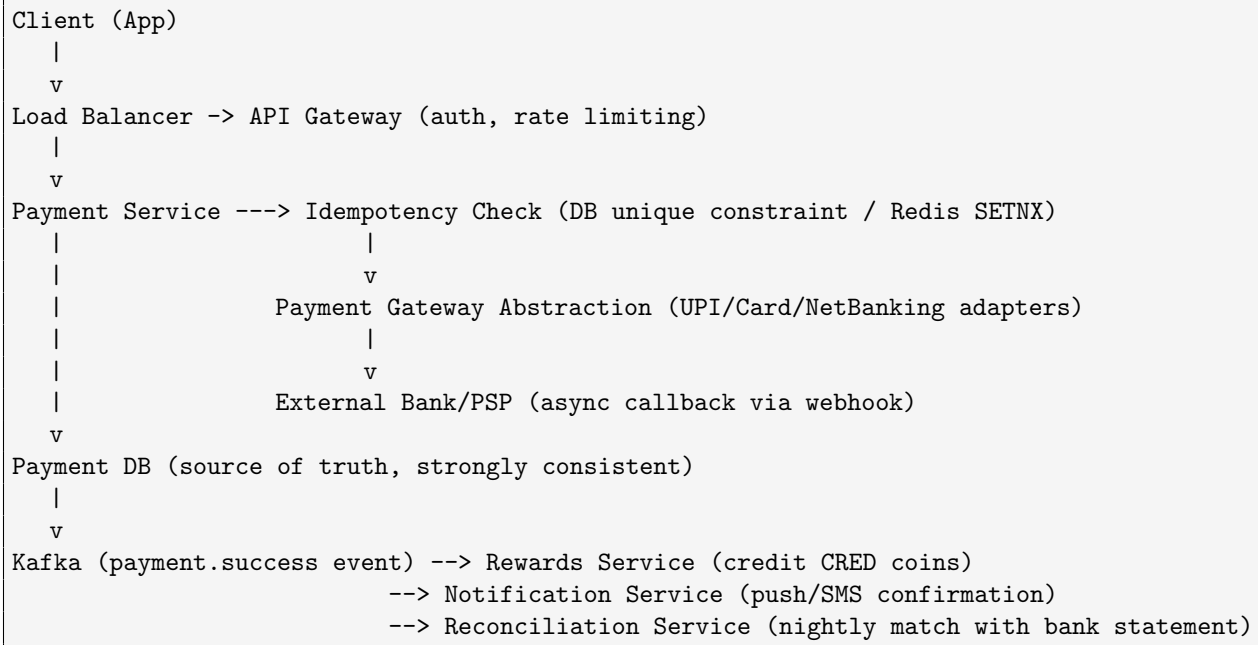
POST /api/v1/payments/webhook/bank-callback (called by bank/PSP, not client)
Body: { paymentId, bankTxnId, status }
```

3.1.5 Database Schema (high level)

- **payments** table (PostgreSQL/MySQL – relational, strong consistency needed): `payment_id` (PK), `idempotency_key` (UNIQUE), `user_id`, `bill_id`, `amount`, `status`, `created_at`, `updated_at`
- **bills** table: `bill_id` (PK), `user_id`, `card_id`, `amount_due`, `due_date`, `status`

- Unique constraint on `idempotency_key` at the DB level – this is the real enforcement point, not just application logic

3.1.6 High-Level Architecture



3.1.7 Deep Dive: Idempotent, Exactly-Once Payment Flow

This is where CRED interviewers push hardest – same theme as LLD 3, now at distributed-systems scale.

1. Client generates a UUID idempotency key locally *before* the first network call, reuses it on every retry of the same logical payment.
2. Payment Service attempts `INSERT` into `payments` with that key as a `UNIQUE` column. If the insert fails on the unique constraint, this is a duplicate – return the existing row's status instead of calling the bank again.
3. Actual bank charge happens *after* this insert succeeds, and the row starts as `PENDING`.
4. Bank confirms asynchronously via webhook (banks don't give synchronous guarantees at scale) – webhook updates status to `SUCCESS/FAILED` using `payment_id`, itself idempotent (re-applying the same webhook is a no-op if status is already terminal).
5. If webhook never arrives (network issue on bank's end), a reconciliation job polls the bank's settlement file nightly and reconciles any `PENDING` payment older than a threshold (out-box/saga pattern).

Trade-offs & Follow-ups

Why DB unique constraint over Redis SETNX for idempotency? → DB constraint is durable and transactional with the payment row itself (atomic with the insert); Redis is faster but needs careful TTL and can lose the key on eviction/crash – CRED-scale money movement favors durability, so DB-level or Redis-with-persistent-AOF-backed.

Why Kafka instead of directly calling Rewards/Notification services? → Decouples payment success from downstream side-effects – if Rewards Service is down, payment still succeeds and coins credit once Rewards catches up from the queue (at-least-once + idempotent consumer).

CAP choice: → Payment write path chooses **Consistency** (single-region strongly consistent DB, synchronous write) even at some availability cost; read path (bill viewing) can be eventually consistent via read replicas for availability.

3.2 HLD 2: Design a De-duplication Service for Duplicate Payments

Difficulty: Hard

3.2.1 Functional Requirements

- Given a stream of incoming payment requests, detect and block duplicates before they hit the payment gateway
- A duplicate = same idempotency key, OR (in absence of a key) same (userId, amount, destination) within a short time window
- Must work as a shared service usable by multiple upstream systems (bill payments, P2P transfers, merchant payments)
- Sub-100ms decision latency (sits in the critical path before charging)

3.2.2 Non-Functional Requirements

- Extremely low latency (in critical path) – favors an in-memory store
- High availability – if the dedup service is down, payments shouldn't halt entirely (need a fallback policy)
- Scale: assume 500 payments/sec average across CRED, peak 2000/sec

3.2.3 Capacity Estimation

- Dedup window of, say, 24 hours: $500/\text{sec} \times 86400 \text{ sec} \approx 43\text{M}$ keys/day resident at once
- Each key entry small (~100 bytes: key + status + timestamp) → $43\text{M} \times 100\text{B} \approx 4.3\text{GB}$ – comfortably fits in a Redis cluster

3.2.4 API Design

```
POST /api/v1/dedup/check-and-reserve
Body: { idempotencyKey, userId, amount, destination }
Response: { isDuplicate: bool, existingStatus?: string }

POST /api/v1/dedup/mark-complete
Body: { idempotencyKey, finalStatus }
```

3.2.5 High-Level Architecture

```

Upstream Service (Bill Payment / P2P / Merchant)
|
v
Dedup Service (stateless, horizontally scaled)
|
v
Redis Cluster (primary store: key -> {status, timestamp}, TTL = window size)
|
v (async write-behind for audit/durability)
Cassandra / DynamoDB (durable log of all dedup decisions, for audits & disputes)

```

3.2.6 Deep Dive: Atomic Check-and-Reserve

The core race condition: two identical requests arrive within milliseconds of each other (client double-tap, or a retry racing the original).

- Use Redis SET key value NX EX <ttl> – this is atomic: it sets the key only if it doesn't already exist, in a single operation. If it returns false, this is a duplicate.
- Composite key design: prefer client-supplied idempotency key when present; fall back to a hash of (userId, amount, destination, rounded-timestamp-to-nearest-10s) when the upstream system doesn't yet send one – this catches accidental double-submits without a formal key.
- TTL matches the business-defined dedup window (e.g., 24h) – after that, an identical payment is treated as legitimately new.

Trade-offs & Follow-ups

Redis is down – what happens to payments? → Fail-safe vs fail-open decision: CRED-scale money movement should **fail closed** conservatively for a few seconds (briefly reject/queue new payments) rather than risk double-charging, but keep the outage window minimal via Redis Sentinel/Cluster failover (usually sub-second).

Why not just rely on the Payment Service's own DB unique constraint (HLD 1)? → That protects against duplicates *within one payment flow*. This standalone Dedup Service protects across *multiple different upstream systems* sharing the same underlying money-movement risk, and does so before the more expensive DB write path is even reached – cheaper, faster rejection.

Why Redis over a DB unique constraint here? → Latency budget (sub-100ms, in critical path) rules out a full ACID DB round-trip at this stage; Redis gives that speed, with Cassandra as the durable audit trail behind it.

3.3 HLD 3: Design a Financial Health / Credit Score Aggregation System

Difficulty: Hard

3.3.1 Functional Requirements

- Pull a user's credit score periodically from multiple bureaus (CIBIL, Experian, Equifax)
- Aggregate spending data from linked bank accounts / cards to compute a "financial health score"

- Show score trend over time, and alert user on significant score drops
- Refresh score on-demand (user pulls to refresh) but rate-limited per bureau's API terms

3.3.2 Non-Functional Requirements

- Read-heavy (users check score often) – favor availability and low read latency
- Eventually consistent is fine (score doesn't need to be real-time to the second)
- External bureau APIs are slow/rate-limited and sometimes unreliable – must isolate their failures from the rest of the app
- Sensitive financial data – encryption at rest, strict access control

3.3.3 Capacity Estimation

- 10M users, assume score refreshed at most once/day per user via scheduled batch $\rightarrow$ 10M bureau calls/day $\approx$ 115 calls/sec sustained – needs to be spread across a rate-limited window per bureau, so actually queued and throttled well below that instantaneous number
- On-demand refresh: assume 5% of DAU (1M) triggers a manual pull/day $\rightarrow$ manageable extra load, but must respect per-bureau rate limits (often 1 request per user per X hours)

3.3.4 API Design

```
GET /api/v1/score/{userId}
Response: { score, bureau, lastUpdated, trend: [...] }

POST /api/v1/score/{userId}/refresh
Response: { status: QUEUED, estimatedTime }

GET /api/v1/score/{userId}/history?range=6m
Response: { dataPoints: [{date, score}, ...] }
```

3.3.5 Database Schema

- **scores** (time-series-friendly store – Cassandra or a time-series DB): `user_id`, `bureau`, `score`, `fetches_at` – partitioned by `user_id`, clustered by `fetches_at` for efficient range queries (trend charts)
- **user_bureau_consent** (relational DB): consent status, last successful pull timestamp – needed for compliance/audit

3.3.6 High-Level Architecture

```
Client -> API Gateway -> Score Read Service --> Cache (Redis, TTL ~1 day) --> Cassandra (scores)
```

```
                                     |
```

```
(cache miss / stale) triggers async refresh
```

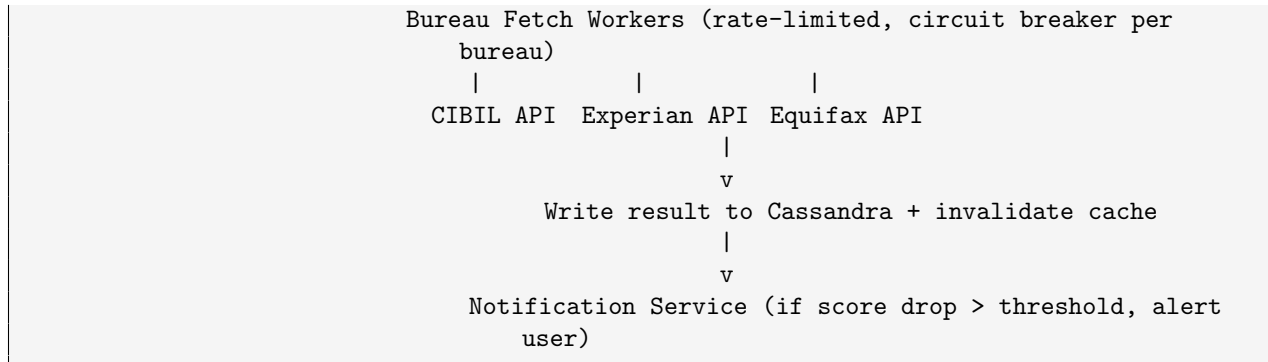
```
                                     |
```

```
                                    v
```

```
                        Score Refresh Queue (Kafka/SQS)
```

```
                                     |
```

```
                                    v
```



3.3.7 Deep Dive: Isolating Unreliable External Bureau APIs

- **Circuit breaker per bureau:** if CIBIL API is failing/slow, trip the breaker so requests fail fast instead of piling up threads/connections waiting – protects the rest of the system from one bureau’s outage.
- **Async worker pool + queue:** refresh requests are never handled synchronously in the request path; user gets QUEUED immediately, actual bureau call happens in a background worker, result pushed via websocket/push notification when ready.
- **Per-bureau, per-user rate limiting:** a token-bucket limiter keyed by (bureau, userId) enforces the bureau’s contractual rate limits, queuing excess requests rather than dropping them.
- **Stale-while-revalidate caching:** always serve the last known score instantly from cache, kick off a background refresh if it’s older than the TTL – keeps reads fast (availability) even though the underlying data is eventually consistent.

Trade-offs & Follow-ups

Why Cassandra over PostgreSQL for scores? → Time-series, write-heavy-ish (daily batch), read pattern is "give me last N scores for this user" – Cassandra’s partition-by-user + clustering-by-time model fits this natively and scales horizontally; a relational DB would need careful indexing and doesn’t scale writes as easily at this shape.

Why not just call the bureau synchronously on every GET /score request? → Violates the rate limit instantly at scale, and ties user-facing latency to the slowest external dependency – classic reason to decouple via cache + async refresh.

Consistency model: → This system deliberately chooses **Availability + eventual consistency** (AP in CAP) since a financial *health score* is directionally useful, not a source of truth for money movement (unlike HLD 1, which must choose CP).

4 Quick Revision Cheat-Sheet

4.1 LLD Pattern Map

- Parking Lot → Strategy (fee calc, spot allocation) + Singleton
- Contact Management → Trie (prefix search) + HashMap (exact lookup)
- Payment Processing → Idempotency Key + atomic check-then-act + Strategy (gateways)
- Rewards System → Decorator/Chain of Responsibility (stacking bonus rules) + idempotency
- Reminder System → Observer (channels) + bucketing by date (avoid full scans) + idempotency log

4.2 HLD Recurring Themes at CRED

- **Idempotency** is the single most repeated theme – client-generated keys + DB unique constraint / Redis atomic SETNX
- **CP vs AP choice** always justified by whether money moves (CP) or not (AP, e.g. score, analytics)
- **Kafka/queue** to decouple the critical payment path from side-effects (rewards, notifications, reconciliation)
- **Circuit breakers** + **async workers** whenever an external, unreliable, rate-limited API is involved
- **Caching with stale-while-revalidate** for anything read-heavy but not real-time critical

4.3 Interview Delivery Tips

- Always start LLD by listing functional requirements out loud and get interviewer buy-in before coding
- Always start HLD with functional + non-functional requirements + a rough capacity estimate before drawing any boxes
- Say the pattern name out loud when you use it ("I'll use Strategy here so we can swap fee logic without touching ParkingLot") – signals intentional design, not luck
- When stuck, ask: "is this idempotency? is this CP or AP? is this a caching problem?" – these three lenses solve most CRED design questions